

Tyler Magarelli

Toronto, ON • (416) 278-5442 • tylermagarelli@gmail.com

SUMMARY

Software engineer with over 5 years of experience shipping production systems, now a design lead on a customer-facing product where I set direction alongside senior leadership and run customer demos. I build production-grade agentic systems end to end, including multi-agent orchestration, model routing, MCP integrations, and event-driven automation triggered from Slack, GitHub, and schedules. I work comfortably across engineering, product, and go-to-market, designing agent workflows alongside users and turning early pilots into production deployments.

SKILLS & CERTIFICATIONS

Languages: Python, Java, C++, C, C#, JavaScript, TypeScript, Go, SQL, R, QML, HTML

AI & Agentic: Multi-agent orchestration, agent harness design, model routing, MCP servers, tool and function calling, prompt engineering, agent evaluation and observability, RAG and vector stores, Claude Code, Anthropic, OpenAI, and local (Ollama) model APIs

Infra & Tooling: Docker, CI/CD with GitHub Actions, Linux, systemd, AWS, Vercel, event-driven automation (Slack, GitHub, webhooks, cron), observability, Git, Jira, Confluence, Jenkins, Agile

Web: React, Next.js, Node, FastAPI, Postgres, Supabase, Redis, Celery, Firebase

Certifications: AWS Certified Cloud Practitioner

PROFESSIONAL EXPERIENCE

ReVue Design Lead — Evertz Microsystems Ltd., Burlington, ON

Jan 2026 - Present

- Promoted to Design Lead for ReVue, a replay product built for non-technical users and fan engagement, where I set the design direction and prioritize the roadmap in direct partnership with the VP of Software.
- Run customer-facing demos from early discovery through technical deep dives, presenting the platform and advising clients on how it fits into their existing architectures.
- Own features end to end, from bug fixes to experimental capabilities, working in an agile environment alongside 4-6 developers on connected platforms and feeding customer feedback back into the roadmap.

Software Design Engineer — Evertz Microsystems Ltd., Burlington, ON

May 2023 - Dec 2025

- Core member on a team of 8-12 developers working towards quarterly releases of a broadcast multiviewer UI system (C++ backend, QML frontend) in an agile development environment.
- Worked directly with customers to validate requested features against their real-world use cases for releases and fixes.
- Helped plan releases and schedule testing across two weekly team meetings with product-facing colleagues.

Software Development Intern — Evertz Microsystems Ltd.

Summer 2021 - Summer 2022

- Developed computer vision software in C++ and Python to locate and identify elements of interest in real-time video and parse data for a machine learning replay system.
- Prototyped with ffmpeg to extract raw RGB data and built image processing algorithms to maximize accuracy.
- Built a web-based graphical telestration tool on Chromium Embedded Framework, creating animated vector overlays delivered through WebSocket servers.

Software Interface Design Intern — Evertz Microsystems Ltd.

Summer 2020

- Developed new UI features in QML, C++, and JavaScript for a Qt-based broadcast multiviewer on a biweekly release cycle, using Jira and Git for issue tracking and version control.

SELECTED PROJECTS

Jarvis — Cloud agent orchestration platform for autonomous software engineering

- Built a platform that dispatches a pipeline of 15 specialized agents (plan, architect, implement, review, test, secure, and ship) to carry out real engineering work across multiple projects, triggered from Slack, GitHub, and schedules and opening pull requests on its own. Built in Python with asyncio, Celery and Redis, Postgres, and a FastAPI dashboard, with 9 opt-in MCP servers and CI enforcing an 80% coverage gate across roughly 15,500 lines of code.
- Engineered a multi-tier model router that maps 14 task types across Anthropic, OpenAI, and local models, with quality-gated escalation, transparent fallback, and per-call logging of tokens, cost, and latency for full observability. Each agent run is scoped to only its credentialed tools through dynamic MCP injection, with separate read-only and write permission modes.
- Designed a self-healing daemon that diagnoses agent and CI failures in an isolated sandbox, gates fixes on lint and tests, and applies them live with health-checked auto-rollback, all bounded by rate caps, cooldowns, and kill switches. These are the same orchestration, sandboxing, and governance concerns enterprise agent platforms depend on.

Scoper — AI job-intake and quoting platform for trade contractors (in active pilot)

- Built a B2B SaaS product where contractors embed an AI chat widget that intakes a job, runs an AI diagnostic, and automatically generates a parts and labor quote. Currently piloted with two private contractors who use it in their daily workflow to harden the platform ahead of a broader sales push. Built with TypeScript, Next.js 15, React 19, Supabase and Postgres, and OpenAI through the Vercel AI SDK, with 26 API routes and 18 migrations deployed on Vercel through GitHub Actions.
- Built a two-stage AI intake that pairs a streaming chat and a validated tool call with a structured diagnostic step returning cause, confidence, severity, and parts, feeding a confidence-adjusted quote calculator with cascading price resolution.
- Implemented field-level PII encryption at the application layer, multi-tenant Postgres row-level security with per-contractor isolation, and sliding-window rate limiting.

Compliance-Aware Real-World-Asset Tokenization Platform (in development)

- Architecting a no-code platform designed to let US and Canadian real-estate syndicators run Reg D 506(c) and NI 45-106 private placements end to end, generating legal documents with AI, deploying compliance-aware security tokens, and onboarding investors with abstracted wallets. Designed around Next.js and TypeScript, Supabase, Hardhat with ERC-3643 security tokens on Polygon, Privy embedded wallets, and Claude for document generation.
- Designed an AI legal-document pipeline with prompt-injection-safe structured input, post-generation consistency validation with reject and retry, mandatory attorney-review markers, and a SHA-256 audit trail for each document.
- Specified an on-chain compliance layer with an identity registry and pluggable rules for investor caps, lockups, and jurisdiction, plus a hard deploy gate that blocks token deployment until documents are approved.

Real-Time Match-Play Golf Tournament Platform

- Built a private real-time SPA serving 16 concurrent users in a two-team match-play tournament, with live score entry, a multi-board leaderboard with live win probability, a captain draft, and encrypted chat, on a Firestore real-time architecture designed to scale well beyond that. Built with React 19, Vite, and Firebase (Firestore, Cloud Functions, anonymous auth), packaged as a PWA.
- Engineered a split-collection real-time architecture, with around 10 Firestore paths recombined by a single selector, using atomic batched writes, optimistic updates, and exponential-backoff retry.
- Built a match-play scoring engine that handles dormie, all-square, and concessions across four formats with USGA handicap allocation, plus a Monte Carlo win-probability model.

LEADERSHIP & COMMUNITY

- Co-founded an internal interest-based community at Evertz with the CTO. It started as a 30-person golf group meant to connect junior engineers, surface new project opportunities, and build cross-team relationships, and has since grown to nearly 500 members across groups spanning gardening, music, volleyball, and improv.
- Part of a 5-person admin team that runs weekly AI-enablement sessions. I have personally led sessions on coding agents and CLI-based agentic tooling such as Claude Code for engineers, and on AI workflows for non-technical staff, helping drive adoption across the company.

INTERESTS

Avid golfer and lifelong multi-sport athlete (ex-AA hockey, still active in football, basketball, hockey, and volleyball). Part-time tutor and mentor with a strong interest in travel and learning about different cultures.